ReNote

Celestial Moon Collection & Anime Media Hub

The Complete Architecture, Development & Deployment Handbook

A comprehensive, end-to-end technical manual and operational guide for building, configuring, styling, and deploying ReNote to production on Render using Django 5, Supabase PostgreSQL, WhiteNoise static compression, and resilient multi-provider API cascading.

Project:	ReNote (Celestial Dark Anime Tracker)
Author / Engineer:	Venz Virni Blanza (venzblanz@gmail.com)
Live URL:	https://renote-c16n.onrender.com/
Repository:	https://github.com/venzblanz/ReNote.git (branch: testing)
Core Stack:	Python 3.12, Django 5.2, PostgreSQL (Supabase), Gunicorn 26.2, WhiteNoise 6.12
Hosting Platform:	Render Cloud Web Service (Linux Docker container)
Publication Date:	November 2026

Handbook Purpose & Scope

This document is an exhaustive educational manual designed to walk through every single technical decision, command, file structure, configuration variable, and bug resolution implemented throughout the ReNote project lifecycle. Whether you are replicating this architecture from scratch or maintaining the production instance, this handbook provides the blueprint.

Table of Contents

Chapter 1: The Vision & Architectural Foundation Page 3
- Philosophy, System Architecture & The High-Level Flow
- Full Directory Structure & File Inventory

Chapter 2: Python Environment & Dependency Isolation Page 4
- Virtual Environments (venv) Demystified
- Exact PowerShell Activation & Package Pinning

Chapter 3: Django Modular Application Architecture Page 5
- The startproject Dot Syntax Trap (renote/renote vs renote/)
- The 5 Core Apps: home, library, login, register, profile/settings

Chapter 4: Database Modeling, PostgreSQL & Migrations Page 7
- Relational Models: MediaCatalog vs UserMediaLog
- Why Local SQLite Fails on Render & The Supabase Advantage
- Migrations Step-by-Step (makemigrations, migrate)

Chapter 5: Static Files Architecture — static/ vs staticfiles/ Page 9
- The Core Question: Why Two Static Folders?
- WhiteNoise: How Production Static Serving Works Without Nginx
- Cache-Busting Hashes (base.babc8cf8d8a1.css) & Gzip (.gz) Compression

Chapter 6: External APIs & Resilient Cascading Engine Page 11
- AniList, Kitsu, and Jikan Rate Limits & Outage Risks
- Connection Pooling, Timeouts & SHA-256 Memory Caching

Chapter 7: Security, Environment Variables & Configuration Decoupling Page 13
- Twelve-Factor App Principles: .env vs .env.example
- Solving 400 Bad Request: ALLOWED_HOSTS & CSRF_TRUSTED_ORIGINS
- SMTP Password Reset Configuration with Gmail App Passwords

Chapter 8: Production Deployment on Render Page 15
- Procfile & Gunicorn Pre-Fork Worker Server
- build.sh: Build Scripts & The Unix Executable Permission (+x)
- Superuser Creation Without Paid Shell

Chapter 9: Responsive Mobile UI & Overflow Engineering Page 17
- Post-Mortem 1: CSS Grid minmax Collapse & The 2-Column Grid
- Post-Mortem 2: Viewport Horizontal Overflow & The 'Right-Slide' Space
- Media Detail Sizing Formula & Ambient Hero Spotlight

Chapter 10: Complete Command Reference & Cheatsheet Page 19
- Categorized Reference for all PowerShell & Bash Operations

Chapter 1: The Vision & Architectural Foundation

1.1 Project Overview & Philosophy

ReNote was conceived as a premium, distraction-free personal anime, manga, novel, and film tracker. Built around the celestial aesthetic of the 'Violet Hour'—that fleeting twilight when reality fades into imagination—and the Moon as humanity's ancient keeper of time and memory, ReNote gives fiction enthusiasts a sacred night-sky archive to record, rate, track rewatches, and revisit every work of art that moved them.

Unlike generic tracking platforms cluttered with ads, social bloat, and slow loading times, ReNote focuses on **blazing speed, high visual elegance, privacy, and architectural resilience**. Every external media search is cached into a permanent local relational catalog, meaning zero repeated API hits and zero delays.

1.2 The High-Level System Architecture

ReNote is engineered using a robust, decoupled modern Python web stack designed for cloud container environments:

- **Presentation Layer (Frontend):** Semantic HTML5, pure CSS3 with custom variables, CSS Grid, Flexbox, and lightweight modular vanilla JavaScript. Zero bulky JS framework dependencies (no React or Vue bundles), ensuring instant initial paint.
- **Application Layer (Django 5):** Python's premier enterprise framework implementing the Model-View-Template (MVT) pattern, providing robust session security, CSRF protection, ORM querying, and templating.
- **Static Asset Engine (WhiteNoise):** High-performance middleware that intercepts static file requests directly within the Python WSGI pipeline, applying content hashing, gzip/brotli compression, and immutable HTTP caching headers.
- **WSGI HTTP Server (Gunicorn):** Green Unicorn acts as the production pre-fork worker HTTP server running inside the Render container, managing concurrent requests and delegating to Django's WSGI application.
- **Database Layer (Supabase PostgreSQL):** Managed enterprise PostgreSQL cloud database communicating over encrypted SSL (`sslmode=require`), handling relational integrity, indexing, and persistent catalog storage.
- **External Gateway (Multi-Provider API Engine):** Custom resilient HTTP client cascading between AniList GraphQL, Kitsu REST, and Jikan REST APIs with automatic connection pooling and failover.

1.3 Full Project Directory Structure

The ReNote codebase is organized into clean, single-responsibility domains:

```
ReNote/
■■■ .env # Local secret configuration (GITIGNORED - NEVER COMMIT!)
■■■ .env.example # Safe template of required environment keys
■■■ .gitignore # Excludes venv, .env, __pycache__, and compiled assets
■■■ build.sh # Render build script (pip install, collectstatic, migrate)
■■■ Procfile # Process manager file for Render (web: gunicorn)
■■■ manage.py # Django administrative command CLI
■■■ requirements.txt # Pinned Python package dependencies
■
■■■ renote/ # Project Root & Core Configuration
■ ■■■ __init__.py
■ ■■■ asgi.py # Asynchronous server gateway
■ ■■■ wsgi.py # Synchronous WSGI gateway (used by Gunicorn)
■ ■■■ urls.py # Master URL routing table
■ ■■■ settings.py # Centralized settings (WhiteNoise, DB, Auth, Security)
■
■■■ home/ # App: Landing dashboard & celestial hero spotlight
■■■ library/ # App: Catalog search, personal media tracker, detail view
■■■ login/ # App: Authentication, sessions, 4-stage password reset
■■■ register/ # App: User registration, terms of service, privacy
■■■ user_profile/ # App: User profile, stats overview, avatars
■■■ user_settings/ # App: Account preferences & settings
■
■■■ static/ # SOURCE static assets (CSS, JS, images, fonts)
■ ■■■ css/ # Uncompressed developer stylesheets
■ ■■■ js/ # Modular frontend scripts
■ ■■■ images/ # Moon assets, avatars, branding logos
■
■■■ staticfiles/ # COMPILED static distribution (created by collectstatic)
■ ■■■ ... # WhiteNoise hashed (.babc8cf8.css) & gzip (.gz) files
■
■■■ templates/ # Django HTML Templates
■■■ shared/ # Base layouts, headers, SEO meta partials
■■■ home/ # Dashboard templates
■■■ library/ # Library list, catalog search, media detail
■■■ login/ # Login, password reset email templates
■■■ user_profile/ # Profile overview templates
```

Chapter 2: Python Environment & Dependency Isolation

2.1 What is a Virtual Environment and Why is it Essential?

In Python development, installing libraries globally across your operating system using `pip install ` is a dangerous anti-pattern. If Project A requires `Django==4.2` and Project B requires `Django==5.2`, installing them globally leads to version collisions, breaking existing applications. Furthermore, global installations pollute your system Python and make it impossible to know which exact packages your project actually depends on.

A **Virtual Environment** (`venv`) is a self-contained directory tree that contains its own Python interpreter, standard libraries, and an isolated `site-packages` directory. When you install packages inside a virtual environment, they are stored exclusively within that folder and cannot affect any other project on your machine.

2.2 Step-by-Step PowerShell Lifecycle

Here are the exact Windows PowerShell commands used to create, activate, and manage the ReNote virtual environment:

```
# 1. Create a dedicated virtual environment named 'vvv'
python -m venv vvv

# 2. Activate the virtual environment in PowerShell
.\vvv\Scripts\activate

# Note: If PowerShell displays an ExecutionPolicy restriction error, run:
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

# 3. Direct invocation without activation (used by IDEs and automated scripts):
.\vvv\Scripts\python.exe manage.py <command>
.\vvv\Scripts\pip.exe install <package>
```

2.3 Pinned Dependencies (`requirements.txt`) Explained

The `requirements.txt` file lists every third-party package necessary to run ReNote in both development and production. Here is what each pinned dependency accomplishes:

Package	Version	Purpose & Role in ReNote
Django	5.2.17	The core web framework handling routing, models, templates, and security.
gunicorn	26.2.0	Green Unicorn: production WSGI HTTP server executing Python worker processes on Render.
whitenoise	6.12.0	Production static asset engine providing hashing, compression, and HTTP caching.
psycopg2-binary	2.9.12	PostgreSQL database adapter allowing Python to communicate natively with Supabase.
dj-database-url	3.1.2	Parses standard DATABASE_URL connection strings into Django dictionary format.
python-dotenv	1.2.3	Loads key-value pairs from a local .env file into os.environ automatically.
requests	2.34.2	Battle-tested HTTP library used to query external anime APIs (AniList, Kitsu, Jikan).
urllib3	2.8.0	Provides connection pooling, retry adapters, and timeout controls for HTTP requests.

pillow	12.3.0	Python Imaging Library used for avatar upload processing and image handling.

Exporting Installed Packages

Whenever you install a new package into your environment, always synchronize requirements.txt by running: .\vvv\Scripts\python.exe -m pip freeze > requirements.txt. This ensures Render installs the exact same package versions during its build step.

Chapter 3: Django Modular Application Architecture

3.1 The `startproject` Dot Syntax Trap

When initializing a new Django project, running `django-admin startproject renote` creates an annoying double-nested folder structure: `renote/renote/settings.py` with `renote/manage.py`. This extra folder confuses Git repositories, Docker containers, and deployment platforms.

By appending a single dot (`.`) to the command: `django-admin startproject renote .`, Django initializes the project directly inside the current root folder. This places `manage.py` at the root and keeps configuration clean.

```
# The Clean Command (Used in ReNote):
django-admin startproject renote .

# Result:
# ./manage.py
# ./renote/settings.py
# ./renote/urls.py
# ./renote/wsgi.py
```

3.2 The 5 Domain Applications of ReNote

In Django, projects are broken down into small, modular 'apps' that each handle one specific business domain. ReNote is divided into five dedicated applications:

- **home (templates/home/, static/css/home/):** Renders the main dashboard, the signature celestial purple moon hero showcase, interactive random title picker modal ('Pick for Me'), format breakdown charts, and collection summaries.
- **library (templates/library/, static/css/library/):** The core media catalog. Houses catalog search, external API search queries, personal collection filtering (Anime, Manga, Movie, Novel), media detail view, status toggle (Watching, Completed, Plan to Watch, On Hold, Dropped), star rating, rewatch counter, and personal review notes.
- **login (templates/login/):** Manages user authentication, secure session cookies, login forms, logout endpoints, and the full 4-stage SMTP password reset workflow.
- **register (templates/register/):** User signup validation, username uniqueness verification, terms of service, and privacy policy views.
- **user_profile & user_settings:** Account customization, avatar selections, personal statistics recalculations, and password change management.

3.3 Master URL Routing Table (`renote/urls.py`)

Django uses a centralized routing table in `renote/urls.py` that delegates URL prefixes to app-specific `urls.py` files. Here is the master routing configuration for ReNote:

```
from django.contrib import admin
from django.urls import path, include
from django.views.generic import TemplateView

urlpatterns = [
path('admin/', admin.site.urls),
path('', include(('home.urls', 'home'), namespace='home')),
path('library/', include(('library.urls', 'library'), namespace='library')),
path('login/', include(('login.urls', 'login'), namespace='login')),
path('register/', include(('register.urls', 'register'), namespace='register')),
path('profile/', include(('user_profile.urls', 'user_profile'), namespace='user_profile')),
path('settings/', include(('user_settings.urls', 'user_settings'), namespace='user_settings')),

# Dynamic SEO Endpoints
path('robots.txt', TemplateView.as_view(template_name='robots.txt', content_type='text/plain')),
path('sitemap.xml', TemplateView.as_view(template_name='sitemap.xml', content_type='application/xml')),
]
```

Chapter 4: Database Modeling, PostgreSQL & Migrations

4.1 Relational Architecture: MediaCatalog vs UserMediaLog

A critical engineering decision in ReNote was separating **canonical media data** from **user personal data**. If 50 users add 'Solo Leveling' to their libraries, saving the title, synopsis, poster image, and release date 50 separate times in the database is wasteful and slow. Instead, ReNote employs a normalized two-table architecture:

- **MediaCatalog (Canonical Entity):** Stores unique external media objects fetched from AniList or Kitsu. Holds `external_id`, `media_title`, `media_type`, `media_description`, `media_image` URL, `media_year`, `media_rate`, and `genre` list. Once saved, any user who adds this title references this exact record. Repeated external API calls drop to ZERO.
- **UserMediaLog (Personal Relationship):** Links a `User` foreign key to a `MediaCatalog` foreign key with a `unique_together = ('user', 'media')` constraint. Stores the user's private data: `status` (Watching/Completed/etc.), `personal_rate` (1 to 5 stars), `rewatch_count`, `media_note` (review/thoughts), `is_favorite` boolean, `created_at`, and `updated_at`.

4.2 The Python Model Schema (`library/models.py`)

```
from django.db import models
from django.contrib.auth.models import User

class MediaCatalog(models.Model):
external_id = models.CharField(max_length=120, blank=True, null=True, db_index=True)
media_type = models.CharField(max_length=30, choices=[
('anime', 'Anime'), ('manga', 'Manga'), ('movie', 'Movie'), ('novel', 'Novel')
], default='anime')
media_title = models.CharField(max_length=255, db_index=True)
media_description = models.TextField(blank=True, default='')
media_image = models.URLField(max_length=600, blank=True, default='')
media_release_date = models.DateField(blank=True, null=True)
media_rate = models.DecimalField(max_digits=3, decimal_places=1, blank=True, null=True)
genre = models.JSONField(default=list, blank=True)

class UserMediaLog(models.Model):
STATUS_CHOICES = [
('watching', 'Watching'), ('reading', 'Reading'),
('completed', 'Completed'), ('plan to watch', 'Plan to Watch'),
('plan to read', 'Plan to Read'), ('on hold', 'On Hold'), ('dropped', 'Dropped')
]
user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='media_logs')
media = models.ForeignKey(MediaCatalog, on_delete=models.CASCADE, related_name='user_logs')
status = models.CharField(max_length=30, choices=STATUS_CHOICES, default='watching')
personal_rate = models.PositiveSmallIntegerField(blank=True, null=True) # 1 to 5 stars
rewatch_count = models.PositiveIntegerField(default=0)
media_note = models.TextField(blank=True, default='')
is_favorite = models.BooleanField(default=False)
updated_at = models.DateTimeField(auto_now=True)

class Meta:
unique_together = ('user', 'media')
```

4.3 Why Local SQLite Fails on Render & The Supabase Advantage

By default, Django creates a local `db.sqlite3` file. While SQLite is fantastic for local development, **deploying SQLite to Render or any modern cloud container platform leads to catastrophic data loss**.

Render free/starter web services operate on **ephemeral container filesystems**. Every time Render deploys a new git commit, restarts an idle container, or spins up a new instance, the container is completely wiped and rebuilt from the git repository. Any SQLite database file written to disk is immediately erased!

To solve this, ReNote connects to an external **Supabase PostgreSQL database**. Supabase provides permanent, independently hosted enterprise PostgreSQL that never sleeps or deletes your tables during Render redeploys.

Chapter 5: Static Files Architecture — static/ vs staticfiles/

5.1 The Core Question: Why Are There Both `static/` and `staticfiles/` Folders?

This is one of the most common questions in Python web engineering. Here is the definitive distinction:

- **static/ = YOUR SOURCE CODE (Developer Input):** This is where you write your original CSS stylesheets, JavaScript files, images, and fonts. You edit files in `static/` every day. They are organized logically by app (`static/css/home/home.css`, `static/js/library/add_title.js`). This folder is committed to Git.
- **staticfiles/ = COMPILED PRODUCTION DISTRIBUTION (Build Output):** This folder is generated automatically when Django runs `python manage.py collectstatic`. Django crawls every app and your `static/` directory, copies all assets into this single unified folder, and passes them to WhiteNoise to be compressed, hashed, and pre-packaged for production.

Golden Rule of Static Files
NEVER manually edit files inside the staticfiles/ folder! Any manual change in staticfiles/ will be permanently overwritten and deleted the next time collectstatic runs. Always edit in static/, and let Django compile into staticfiles/.

5.2 Why Django Refuses to Serve Static Files in Production (`DEBUG = False`)

When `DEBUG = True` (development mode), Django's internal development server (`runserver`) intercepts requests for `/static/...` and serves files dynamically. However, the moment you flip `DEBUG = False` for production, Django's static file handler **completely shuts down**. Why? Because serving static files through a Python web framework is slow and memory-intensive; Python is designed for business logic and database transactions, not reading bytes off a disk.

Historically, developers were forced to configure a separate Nginx web server or upload assets to an Amazon AWS S3 bucket. This was notoriously complex for modern container platforms like Render, Heroku, or Fly.io.

5.3 WhiteNoise: The Zero-Configuration Production Static Engine

WhiteNoise revolutionizes static serving in Python. By adding a single middleware line (`whitenoise.middleware.WhiteNoiseMiddleware`) into `settings.py`, WhiteNoise embeds static serving directly into Gunicorn's WSGI pipeline with performance comparable to Nginx.

5.4 Content Hashing (`base.babc8cf8d8a1.css`) & The Cache-Busting Mechanism

ReNote configures WhiteNoise's `CompressedManifestStaticFilesStorage`. When `collectstatic` runs, WhiteNoise computes a cryptographic SHA-1 hash of each file's contents and embeds it into the filename: `base.babc8cf8d8a1.css`. Why?

This solves the dreaded **stale browser cache problem**. In production, WhiteNoise sets the HTTP header Cache-Control: max-age=31536000, immutable. Browsers cache this CSS file on the user's phone for an entire year, making page loads instantaneous. When you release a new CSS update, its hash changes to `base.70367f7.css`. The browser immediately detects the new filename and downloads the fresh CSS without users ever needing to clear their cache!

5.5 Why `.gz` (Gzip) and `.br` (Brotli) Files Exist in `staticfiles/`

Inside `staticfiles/`, you will notice files like `base.css.gz` alongside `base.css`. WhiteNoise pre-compresses your CSS and JS at build time using Gzip and Brotli. When a mobile phone requests `base.css` with an `Accept-Encoding: gzip` header, WhiteNoise serves the pre-compressed byte stream directly off the disk. This shrinks asset sizes by **70% to 80%** with **zero runtime CPU overhead** on your Render server!

Chapter 6: External APIs & Resilient Cascading Engine

6.1 The Fragility of Third-Party Anime APIs

ReNote allows users to search across millions of anime, manga, movies, and light novels. To power this, the app connects to public APIs including AniList, Kitsu, and Jikan (MyAnimeList). However, relying on third-party APIs in production presents massive risks:

- **Rate Limiting (HTTP 429 Too Many Requests):** Public APIs strictly limit requests per minute. If multiple users search simultaneously, the API blocks the server.
- **Slow Network Spikes & Freezes:** When an external API experiences heavy load, requests hang for 30+ seconds, causing Gunicorn worker timeout errors.
- **Cloudflare Captchas & Outages:** Free APIs frequently undergo maintenance, downtime, or Cloudflare bot challenges.

6.2 ReNote's 4-Layer Resilient Cascading Architecture

To guarantee that ReNote NEVER crashes or hangs during a catalog search, a custom 4-layer cascading engine was built in `library/utils.py`:

- **Layer 1: Unified Connect & Read Timeouts (3.0s, 5.0s):** Every external request is strictly bounded. If a provider fails to establish a TCP handshake within 3.0 seconds or return bytes within 5.0 seconds, the request is instantly aborted and failed over to the next provider.
- **Layer 2: HTTP Connection Pooling & Retries:** Uses urllib3.util.Retry with an exponential backoff factor (0.5s) on transient HTTP status codes (500, 502, 503, 504), reusing established TLS connections.
- **Layer 3: Provider Failover Waterfall:** Searches follow a strict priority waterfall: AniList (GraphQL) -> Kitsu (REST) -> Jikan (MyAnimeList) -> Local Database. If AniList is down or rate-limited, Kitsu takes over instantly with zero user interruption.
- **Layer 4: In-Memory SHA-256 Request Cache:** Every search query is hashed using SHA-256 and stored in memory with an expiration TTL. If two users search 'Frieren', the second request returns in 1 millisecond with zero external network traffic.

6.3 The Resilient Engine Implementation (`library/utils.py`)

```
import requests
from urllib3.util import Retry
from requests.adapters import HTTPAdapter

def get_resilient_session():
session = requests.Session()
# Retry on server errors with exponential backoff (0.5s, 1.0s, 2.0s)
retries = Retry(total=2, backoff_factor=0.5, status_forcelist=[500, 502, 503, 504])
adapter = HTTPAdapter(max_retries=retries, pool_connections=10, pool_maxsize=20)
session.mount('https://', adapter)
session.mount('http://', adapter)
return session

def search_media_cascade(query, media_type='anime'):
session = get_resilient_session()
timeouts = (3.0, 5.0) # (Connect timeout, Read timeout)

# 1. Try AniList GraphQL
try:
results = search_anilist(session, query, media_type, timeout=timeouts)
if results: return results
except Exception:
pass # Fail over to Kitsu

# 2. Try Kitsu REST API
try:
results = search_kitsu(session, query, media_type, timeout=timeouts)
if results: return results
except Exception:
pass # Fail over to Jikan

# 3. Try Jikan (MyAnimeList)
try:
return search_jikan(session, query, media_type, timeout=timeouts)
except Exception:
# 4. Fallback to local database cache
return search_local_catalog(query, media_type)
```

Chapter 7: Security, Environment Variables & Configuration Decoupling

7.1 Twelve-Factor App Principles: `.env` vs `.env.example`

Modern cloud security mandates **strict separation of code from configuration**. Sensitive credentials (database passwords, secret encryption keys, SMTP email credentials) must NEVER be hardcoded inside Python files. If a repository with hardcoded credentials is pushed to GitHub, automated bot crawlers scrape the keys within seconds.

ReNote utilizes `python-dotenv` to decouple configuration across two distinct files:

- **.env (PRIVATE SECRETS - GITIGNORED):** Contains your actual production passwords and keys. This file is listed in `.gitignore` and NEVER leaves your local development machine or Render's encrypted environment variables dashboard.
- **.env.example (PUBLIC TEMPLATE - COMMITTED):** Contains dummy placeholder keys (`DATABASE_URL=postgresql://user:pass@host:5432/db`). It acts as documentation for other developers so they know exactly which environment variables the app requires to boot.

7.2 Resolving `400 Bad Request` (`ALLOWED_HOSTS` & `CSRF_TRUSTED_ORIGINS`)

When ReNote was first deployed to Render at `https://renote-c16n.onrender.com/`, the server returned an immediate **HTTP 400 Bad Request (DisallowedHost)**. Here is why this happened and how it was solved:

Django includes built-in security against **HTTP Host Header Poisoning attacks**. If an incoming HTTP request arrives with a `Host` header that is not explicitly registered in `ALLOWED_HOSTS`, Django immediately terminates the request with a 400 status. Because Render assigns random subdomains (e.g. `renote-c16n.onrender.com`), hardcoding `renote.onrender.com` caused all requests to fail.

The solution was implementing a resilient wildcard parser in `renote/settings.py`:

```
# Resilient Host & CSRF Configuration in renote/settings.py
ALLOWED_HOSTS = ['localhost', '127.0.0.1', '.onrender.com', '*']

# CSRF Trusted Origins for HTTPS form submissions on Render
CSRF_TRUSTED_ORIGINS = [
'https://*.onrender.com',
'https://renote-c16n.onrender.com',
'http://localhost:8000',
'http://127.0.0.1:8000'
]

# Inform Django that Render's reverse proxy terminates HTTPS
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
```

7.3 SMTP Password Reset Configuration (Gmail App Passwords)

ReNote implements the complete 4-stage Django password reset workflow (`password_reset`, `password_reset_done`, `password_reset_confirm`, `password_reset_complete`) sending multipart HTML emails styled in ReNote's signature purple palette. Here is the production SMTP email backend configured in `renote/settings.py`:

```
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = os.getenv('EMAIL_HOST_USER')
EMAIL_HOST_PASSWORD = os.getenv('EMAIL_HOST_PASSWORD') # 16-character Google App Password
DEFAULT_FROM_EMAIL = f'ReNote <{EMAIL_HOST_USER}>'
```

Chapter 8: Production Deployment on Render

8.1 The Anatomy of Render Web Services

Render is a modern cloud hosting platform that runs containerized web services. When you push a git commit to GitHub, Render triggers an automated two-phase deployment pipeline:

- **1. The Build Phase (`build.sh`):** Render launches a Linux build container, clones your repository, installs Python packages, compiles static assets with WhiteNoise, and applies database migrations.
- **2. The Run Phase (`Procfile`):** Once the build completes, Render boots the runtime container and starts Gunicorn to listen on port 10000, routing public HTTPS traffic to your app.

8.2 The Build Script (`build.sh`) Line-by-Line

Here is the exact production build script executed by Render:

```
#!/usr/bin/env bash
# Exit immediately if any command returns a non-zero exit status (prevents broken builds)
set -o errexit

# 1. Install all pinned Python dependencies into the container
pip install -r requirements.txt

# 2. Collect and compress all static files (WhiteNoise hashing & .gz generation)
python manage.py collectstatic --noinput

# 3. Apply all database migrations to Supabase PostgreSQL
python manage.py migrate
```

8.3 The Unix Executable Permission Bug (`chmod +x`)

On Windows operating systems, files created via PowerShell or standard editors have Windows file attributes (`mode 100644`). However, Render runs on **Linux (Ubuntu)**. When Render attempted to execute `./build.sh`, the Linux kernel refused with: bash: ./build.sh: Permission denied because the file lacked the Unix execute (`+x`) permission bit.

Because Windows does not have native Unix chmod commands, the fix was executed directly through Git's index:

```
# Update Git's internal index to set Unix executable mode (100755)
git update-index --chmod=+x build.sh

# Commit the permission change
git commit -m "chore(deploy): make build.sh executable"
git push origin testing
```

8.4 The Procfile & Gunicorn Pre-Fork Worker Server

The `Procfile` is a text file that declares the command used to start the web service. In ReNote, the Procfile contains:

```
web: gunicorn renote.wsgi:application
```

Gunicorn (Green Unicorn) uses a master-worker process model. The master process monitors worker processes. Each worker process handles incoming HTTP requests concurrently, delegating to `renote.wsgi:application`. Render dynamically sets `WEB_CONCURRENCY=1` based on the available container CPU cores to prevent memory exhaustion.

8.5 Managing Superusers Without a Paid Interactive Shell

On Render's free tier, the interactive 'Shell' tab is disabled. To create an admin superuser without shell access, developers have two sanctioned, reliable methods:

Method A: Automated Python Management Script: Create a one-off management command or run a Python snippet that checks `User.objects.filter(is_superuser=True).exists()` and creates one if missing during `build.sh`.

Method B: Direct Supabase SQL Editor: Connect to your Supabase project dashboard, open the SQL Editor, and insert or update `is_superuser = true, is_staff = true` on your existing registered account.

Chapter 9: Responsive Mobile UI & Overflow Engineering

9.1 Post-Mortem 1: The 1-Card-Per-Screen Grid Collapse

The Symptom: On mobile phones, library cards and search results took up the entire screen width, displaying only one giant poster per screen height and forcing exhausting, endless scrolling.

The Root Cause: The grid used `grid-template-columns: repeat(auto-fill, minmax(180px, 1fr))`. On a smartphone screen (typically 360px to 412px wide), after subtracting 32px of page padding, the available width was ~328px. Fitting two 180px columns required at least `180 * 2 + 20 (gap) = 380px`. Because 380px > 328px, CSS Grid collapsed into a single 100% wide column. Because anime posters use a `2:3` aspect ratio, a 328px wide card expanded to **~500px in height**, filling the entire screen for a single item!

The Engineering Fix: In `@media (max-width: 640px)`, we strictly enforced a 2-column grid:

```
@media (max-width: 640px) {
.library-grid, .results-grid {
grid-template-columns: repeat(2, minmax(0, 1fr)) !important;
gap: 10px !important;
}
.media-card, .result-card {
padding: 5px;
border-radius: 12px;
}
.card-body {
padding: 8px 5px 6px;
}
.card-title {
font-size: 0.85rem;
line-height: 1.25;
height: 2.5em;
min-height: 2.5em;
max-height: 2.5em;
}
}
```

9.2 Post-Mortem 2: Viewport Horizontal Overflow & The 'Right-Slide' Space

The Symptom: On the dashboard and media detail pages, users could slide/pan the entire page horizontally to the right, revealing a massive dead gutter, while the Library and Add Title pages stayed firmly locked to the screen edge.

The Root Cause (Deep Forensic Analysis):

- **Dashboard Footer Blowout:** `.dash-footer` had 7 links (`Dashboard`, `Library`, `Search`, etc.) and the copyright text in a single row with `gap: 20px` and NO `flex-wrap`. This single row measured **~850px wide**, forcing the entire mobile document to expand to 850px! Library and Add Title didn't have this bug because they don't use `.dash-footer`.
- **Backdrop Banner Scale:** `.detail-backdrop-banner` on media details had `transform: scale(1.08)` spanning `left: 0; right: 0;`. This pushed the banner 8% beyond the right edge of the screen.
- **Missing Global Bounding:** Neither `html` nor `body` had `overflow-x: hidden; max-width: 100vw;`, allowing overflowing elements to expand the mobile viewport canvas.

The Engineering Fix:

```
/* 1. Global Viewport Bounding in base.css */
html, body {
width: 100%;
max-width: 100vw;
overflow-x: hidden;
position: relative;
}

/* 2. Responsive Wrapped Footer in home.css */
@media (max-width: 900px) {
.dash-footer {
flex-direction: column;
align-items: center;
gap: 16px;
padding: 24px 16px;
width: 100%;
max-width: 100vw;
box-sizing: border-box;
overflow-x: hidden;
}
.footer-links {
flex-wrap: wrap;
justify-content: center;
gap: 12px 18px;
}
}

/* 3. Media Details Bounding in media_detail.css */
@media (max-width: 900px) {
.detail-backdrop-banner {
transform: none !important; /* Eliminate 8% scale protrusion */
width: 100%;
max-width: 100vw;
overflow: hidden;
}
}
```

Chapter 10: Complete Command Reference & Cheatsheet

This chapter compiles every single command used throughout the construction, testing, static compilation, and deployment of ReNote into an categorized reference table:

Category	Command Line	Technical Explanation & Purpose
Environment	python -m venv vvv	Creates an isolated Python virtual environment named 'vvv' with its own interpreter and site-packages.
Environment	.\vvv\Scripts\activate	Activates the virtual environment in PowerShell, prepending `(vvv)` to the shell prompt.
Dependencies	pip install -r requirements.txt	Reads requirements.txt and installs all exact pinned packages into the active environment.
Dependencies	pip freeze > requirements.txt	Exports all currently installed package versions into requirements.txt for Render synchronization.
Django Init	django-admin startproject renote .	Initializes Django in the current directory with manage.py at root (avoiding nested folder traps).
Django App	python manage.py startapp <app>	Creates a new modular Django application directory with models, views, and admin files.
Database	python manage.py makemigrations	Inspects models.py and generates Python migration blueprint files describing schema changes.
Database	python manage.py migrate	Applies pending migration blueprints against PostgreSQL, executing SQL table creations and alterations.
Diagnostics	python manage.py check	Performs a complete validation of settings, models, routing, and configurations without running the server.
Static Engine	python manage.py collectstatic --noinput	Crawls static/ sources, compiles them into staticfiles/, hashes filenames, and creates .gz pre-compressed assets.
Local Server	python manage.py runserver	Starts Django's local development server listening at http://127.0.0.1:8000/ with hot reloading.
Git Staging	git add <files>	Stages modified files into the Git staging area preparing them for the next commit snapshot.
Git Permission	git update-index --chmod=+x build.sh	Sets Unix executable mode 100755 on build.sh directly inside Git's index, bypassing Windows limitations.
Git Commit	git commit --author="Venz Virni Blanza <venzblanz@gmail.com>" -m "<msg>"	Creates a Git commit snapshot attributed to Venz Virni Blanza with a structured commit message.
Git Push	git push origin testing	Pushes committed code to the 'testing' branch on GitHub, automatically triggering a live build on Render.
Production WSGI	gunicorn renote.wsgi:application	Boots the Green Unicorn WSGI HTTP server executing worker processes in production on Render.

Handbook Complete

Congratulations! You now possess a comprehensive, publication-grade manual covering the complete architectural, operational, and engineering lifecycle of ReNote. Store this PDF safely as your master project reference.